

Качество кода

Лекция



Имя Фамилия

«Здесь написать про себя»

Фото спикера

Сегодня вы узнаете, как:

1

Покрыть код unit-тестами

2

Использовать моки

3

Писать тесты для http-обработчиков с помощью
httptest

4

Писать интеграционные тесты

5

Использовать линтеры и форматтеры

План занятия

1

Теория
тестирования

2

Unit
тестирование
и Моки

3

Интеграционные
тесты

4

Линтеры
и codestyle

I. Теория тестирования

- 01 Что такое тесты
- 02 Пирамида тестирования
- 03 TDD

Что такое тесты

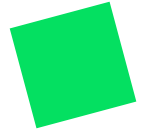
Тест — код, который запускает другой код и сравнивает результат с ожидаемым.



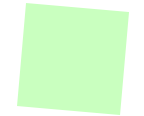
проверяют поведение



ловят регрессии



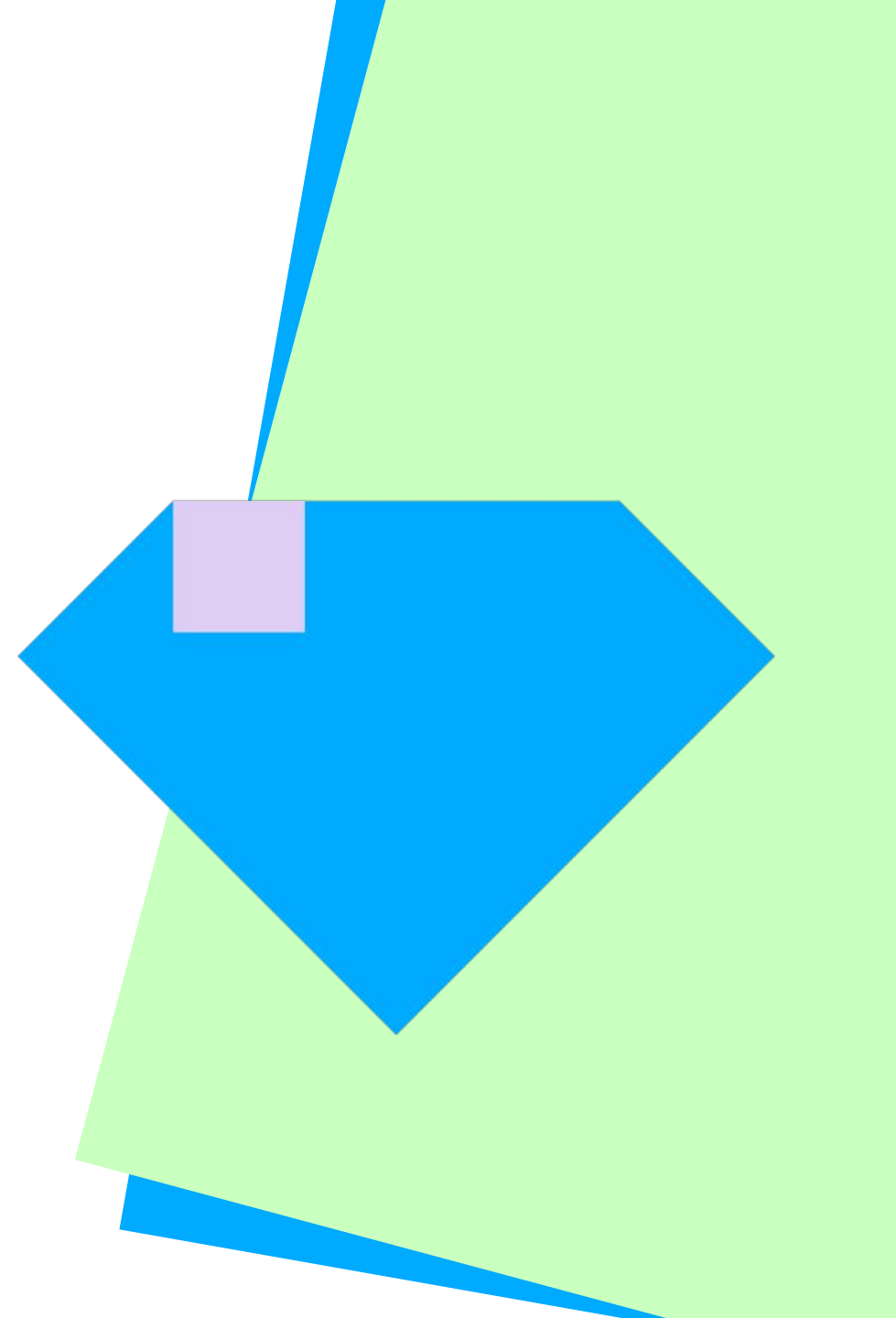
упрощают изменения



описывают требования



ускоряют обратную связь

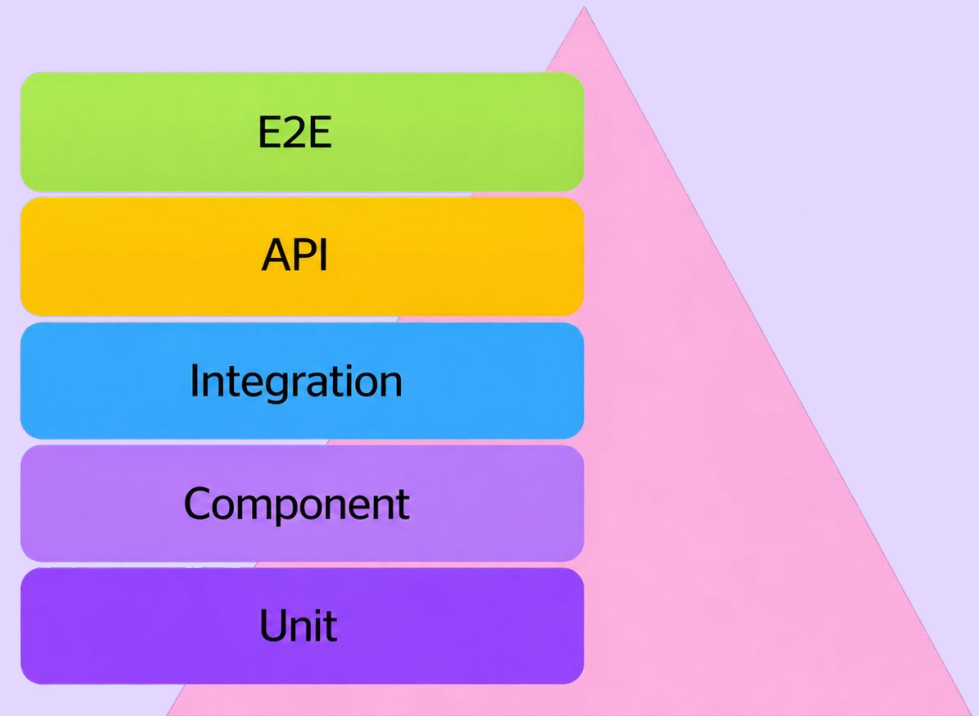


Пирамида тестирования

Чем ниже уровень,
тем больше тестов и тем быстрее
обратная связь.

Быстро и дешево
Unit-тесты запускаем часто

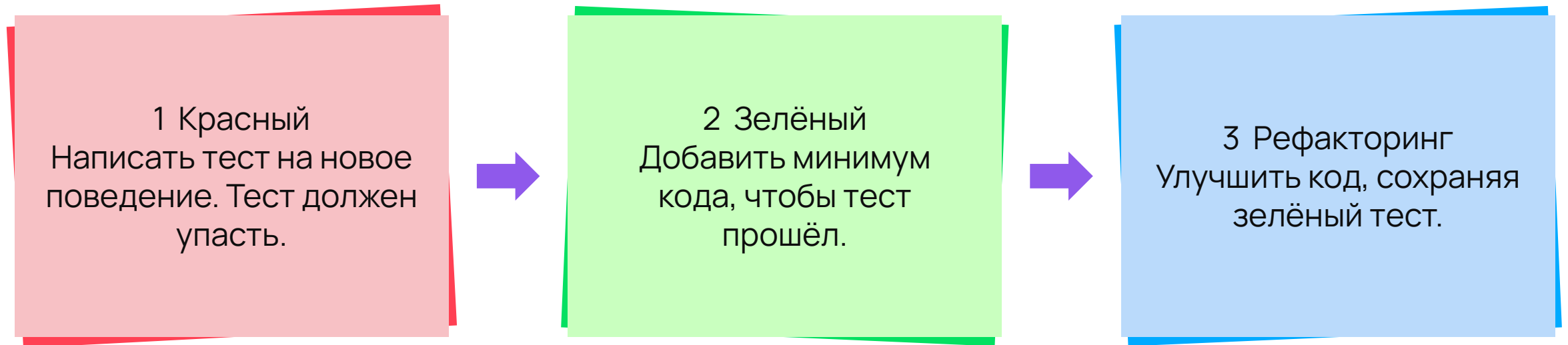
Медленно и дорого
E2E оставляем для ключевых сценариев



Количество тестов уменьшается снизу вверх.

Цикл TDD

Каждое изменение проходит три коротких шага.



Цикл повторяется небольшими шагами.

II. Unit тесты

- 01 Пакет testing
- 02 Табличные тесты
- 03 Пакет testify

Пакет `testing` и команда `go test`

Стандартный пакет Go, предоставляющий всё необходимое для написания и запуска тестов — он не требует сторонних зависимостей.

Команда запуска тестов в проекте
`go test ./...`

Основные функции и соглашения:

- Имя файла: заканчивается на `_test.go`.
- Имя функции: начинается с `TestXxx`, где `Xxx` — имя тестируемой функции (например, `TestAdd`).
- Параметр: `t *testing.T` для управления состоянием теста.
- Запуск: команда `go test`.
- Методы `testing.T`:
 - `t.Error()` / `t.Errorf()` — сообщение об ошибке, но тест продолжается.
 - `t.Fatal()` / `t.Fatalf()` — критическая ошибка, тест немедленно прекращается.
 - `t.Log()` — вывод информации только если тест упал или в `verbose`-режиме (`-v`).

Пакет testing

Запуск:

```
$ go test
```

PASS

```
ok    example/math 0.002s
```

```
import (  
    "testing"  
)  
  
func TestAdd(t *testing.T) {  
    result := Add(2, 3)  
    expected := 5  
  
    if result != expected {  
        t.Errorf("Add(2, 3) = %d; ожидалось %d", result,  
            expected)  
    }  
  
    t.Log("Тест сложения завершен")  
}
```

Табличные тесты

Это паттерн организации тестов, позволяющий тестировать одну функцию с разными наборами входных данных и ожидаемых результатов в структурированной форме.

Для запуска именованных подтестов, внутри основного теста, используется метод пакета `testing` `t.Run()`

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name string
        a    int
        b    int
        want int
    }{
        {"positive numbers", 2, 3, 5},
        {"negative numbers", -1, -1, -2},
        {"mixed signs", 5, -3, 2},
        {"zero case", 0, 0, 0},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Add(tt.a, tt.b); got != tt.want {
                t.Errorf("Add(%d, %d) = %d, want %d",
                    tt.a, tt.b, got, tt.want)
            }
        })
    }
}
```

testify

Популярный сторонний пакет (github.com/stretchr/testify), который предоставляет более удобный синтаксис для утверждений (assertions), организацию тестов в сьюты (suites)

Упрощает написание тестов, делает их более читаемыми и уменьшает количество boilerplate-кода.

```
func TestCalculate(t *testing.T) {  
    result, err := Calculate(2, 3)  
    require.NoError(t, err)  
    assert.Equal(t, 5, result)  
  
    result, err = Calculate(5, 0)  
    require.Error(t, err)  
    assert.Equal(t, 0, result)  
  
    assert.Panics(t, func() {  
        panic("test panic")  
    })  
}
```

httptest

Стандартный пакет для тестирования HTTP-кода: как клиентов, так и серверов. Позволяет тестировать HTTP-обработчики без запуска реального сервера.

```
func TestHelloHandler(t *testing.T) {  
    // Создаем запрос к нашему обработчику  
    req, err := http.NewRequest("GET", "/hello?name=Alice", nil)  
    if err != nil {  
        t.Fatal(err)  
    }  
  
    // Создаем ResponseRecorder (реализует http.ResponseWriter)  
    // для захвата ответа  
    rr := httptest.NewRecorder()  
    handler := http.HandlerFunc(HelloHandler)  
  
    // Вызываем обработчик напрямую  
    handler.ServeHTTP(rr, req)  
  
    // Проверяем статус-код  
    assert.Equal(t, http.StatusOK, rr.Code)  
  
    // Проверяем тело ответа  
    expected := "Hello, Alice!"  
    assert.Equal(t, expected, rr.Body.String())  
}
```

Ваши вопросы

III. Моки

- 01 Что такое моки и зачем они нужны
- 02 Создание моков вручную
- 03 Генерация моков

Что такое мок

Объекты-заглушки, которые имитируют поведение реальных зависимостей (например, других объектов, базы данных, внешнего API). Они программируются на возврат определённых ответов на определённые вызовы.

Чтобы использовать моки, тестируемый метод должен принимать зависимости через интерфейсы, а не через конкретные типы.

```
type Database interface {
    GetUser(id int) (string, error)
}

type Service struct {
    DB Database
}

func (s *Service) GetUserName(id int) (string, error) {
    return s.DB.GetUser(id) // Наша логика зависит от интерфейса, а
                             не от конкретной БД
}

type SimpleDB struct {}

func (db *SimpleDB) GetUser(id int) (string, error) {
    return "Alice", nil
}
```

Генерация моков

Писать моки вручную для сложных интерфейсов — долго и трудно поддерживать, поэтому **mockgen** делает это за нас.

Установка: `go install`
`go.uber.org/mock/mockgen@latest`

Генерация: `mockgen -source=user.go`
`-destination=user_mock.go`
`-package=user`

`-source`: файл с интерфейсом
`-destination`: файл для кода мока
`-package`: пакет для генерируемого файла

```
func TestGetUserName_Success(t *testing.T) {
    ctrl := gomock.NewController(t)

    mockDB := NewMockDatabase(ctrl) // Используем
    // сгенерированный мока
    // Настраиваем ожидание: при вызове GetUser(1) вернуть
    "Alice", nil
    mockDB.EXPECT().GetUser(1).Return("Alice", nil)

    service := &Service{DB: mockDB}
    name, err := service.GetUserName(1)

    if err != nil {
        t.Fatal("unexpected error")
    }
    if name != "Alice" {
        t.Errorf("unexpected name: %s", name)
    }
}
```

Директива go:generate

Директива хранит команду генератора рядом с исходным кодом.

Запуск

go generate ./...

Go выполняет директивы только по этой команде. go build и go test их пропускают.

Генератор запускается из каталога пакета.

```
// repo.go
package user

//go:generate mockgen -source=repo.go -destination=mock.go

type Repository interface {
    GetUser(id int64) (User, error)
}
```

Ваши вопросы

IV. Интеграционные тесты

- 01 Чем отличаются от unit-тестов
- 02 Использование testSuite

Интеграционные тесты

Проверяют взаимодействие между несколькими компонентами системы. Используют реальные внешние зависимости: базы данных, API, файловые системы. Они требуют настройки тестового окружения перед выполнением и выполняются медленнее из-за работы с реальными сервисами

Go позволяет использовать build tags для условной компиляции тестов. `go test -tags=integration ./...`

```
// +build integration
```

```
package integration
```

```
import (  
    "testing"  
    "time"  
    "context"  
)
```

```
// Этот файл будет скомпилирован только при  
указании тега integration
```

```
func TestDatabaseIntegration(t *testing.T) {  
    ctx := context.Background()  
  
    ...  
}
```

Testcontainers

Testcontainers запускает реальный сервис в Docker на время теста.

Контейнер

- Стартует с нужной версией
- Ждёт готовности
- Отдаёт адрес подключения
- Удаляется после теста

Каждый прогон получает отдельный сервис без общей тестовой базы.

```
ctx := context.Background()

pg, err := postgres.Run(
    ctx,
    "postgres:16-alpine",
    postgres.WithDatabase("app"),
    postgres.WithUsername("test"),
    postgres.WithPassword("test"),
    postgres.BasicWaitStrategies(),
)

require.NoError(t, err)
testcontainers.CleanupContainer(t, pg)

dsn, err := pg.ConnectionString(
    ctx, "sslmode=disable",
)

require.NoError(t, err)
```

TestSuite

Тестовый набор — это структурированный подход к организации тестов, который позволяет группировать связанные тесты, разделять общую логику подготовки данных и очистки, а также создавать иерархическую структуру тестов.

```
type UnitTestSuite struct {
    suite.Suite
    mockRepo *MockUserRepository
    service *UserService
}

func (suite *UnitTestSuite) SetupTest() {
    suite.mockRepo = new(MockUserRepository)
    suite.service = NewUserService(suite.mockRepo)
}

func (suite *UnitTestSuite) TestUserServiceUnit() {
    // Настройка моков
    expectedUser := &User{ID: 1, Name: "John"}
    suite.mockRepo.On("FindByID", 1).Return(expectedUser, nil)

    // Тестирование изолированной логики
    user, err := suite.service.GetUser(1)

    suite.NoError(err)
    suite.Equal(expectedUser, user)
    suite.mockRepo.AssertExpectations(suite.T())
}

func TestUnitTestSuite(t *testing.T) {
    suite.Run(t, new(UnitTestSuite))
}
```

Ваши вопросы

V. Качество тестов

- 01 Параллельное выполнение тестов
- 02 Обнаружение гонок, утечек
- 03 Flaky-тесты
- 04 Метрика code coverage
- 05 Мутационное тестирование
- 06 Нагрузочное тестирование

Параллельный запуск тестов

Возможность запускать несколько тестов одновременно для ускорения общего времени выполнения тестового пакета.

Значительное ускорение прогона больших тестовых наборов, особенно интеграционных.

Добавьте `t.Parallel()` в начале теста. Go сам управляет параллельным выполнением.

```
func TestSlowCalculationA(t *testing.T) {  
    t.Parallel() // Этот тест будет запущен параллельно  
                // ... длинная логика  
}  
func TestSlowCalculationB(t *testing.T) {  
    t.Parallel() // ...и этим  
                // ... длинная логика  
}
```

Обнаружение состояния гонки

Go предоставляет встроенный инструмент для обнаружения состояний гонки, который работает на этапе выполнения программы и позволяет выявлять потенциально опасные операции доступа к памяти.

Для запуска детектора гонок нужно добавить флаг `-race` в команду `go test`

```
go test -race ./...
```

Обнаружение утечек горутин

Библиотека `goleak` — инструмент для обнаружения утечек горутин в тестовой среде. Эта библиотека интегрируется со стандартной системой тестирования Go и позволяет автоматически обнаруживать горутин, которые были созданы во время выполнения теста, но не были корректно завершены к его окончанию.

установка `go get go.uber.org/goleak`

Для запуска нужно добавить вызов `defer golek.VerifyNone(t)` в тест

```
import "go.uber.org/goleak"

func TestLeakyFunction(t *testing.T) {
    // Включаем обнаружение утечек для этого теста
    defer golek.VerifyNone(t)

    // Вызываем тестируемую функцию
    processData([]string{"test1", "test2"})

    // Даже после завершения функции горутина
    // продолжает работать
    time.Sleep(2 * time.Second)
    // В этот момент тест обнаружит утечку и
    // завершится с ошибкой
}
```

Задание

1. Что не так с этим тестом?
2. Напишите в чате или проговорите голосом

```
func TestUserValidation(t *testing.T) {  
    rand.Seed(time.Now().UnixNano())  
    userID := rand.Intn(100)  
  
    err := ValidateUser(userID)  
    if err != nil {  
        t.Errorf("Validation failed for user ID %d: %v",  
            userID, err)  
    }  
}
```

Задание

1. Что не так с этим тестом?

+ Правильный ответ
— иногда он падает

```
func TestUserValidation(t *testing.T) {  
    rand.Seed(time.Now().UnixNano())  
    userID := rand.Intn(100)  
  
    err := ValidateUser(userID)  
    if err != nil {  
        t.Errorf("Validation failed for user ID %d: %v",  
            userID, err)  
    }  
}
```

Поиск flaky тестов

Флаки-тесты — это недетерминированные тесты, которые могут проходить или падать при одних и тех же условиях и с одним и тем же кодом. Они представляют серьезную проблему для процесса разработки, поскольку подрывают доверие к тестовой системе, маскируют реальные ошибки и значительно увеличивают время на поддержку тестового набора.

Для того, чтобы обнаружить такие тесты, нужно много раз запустить тесты. Для этого используется флаг `-count`

```
go test -count=100 ./...
```

Некоторые тесты могут проходить только если их запускать в определённом порядке, например, если используют общую переменную. Такие тесты можно обнаружить перемешав порядок запуска тестов

```
go test -shuffle=on ./...
```

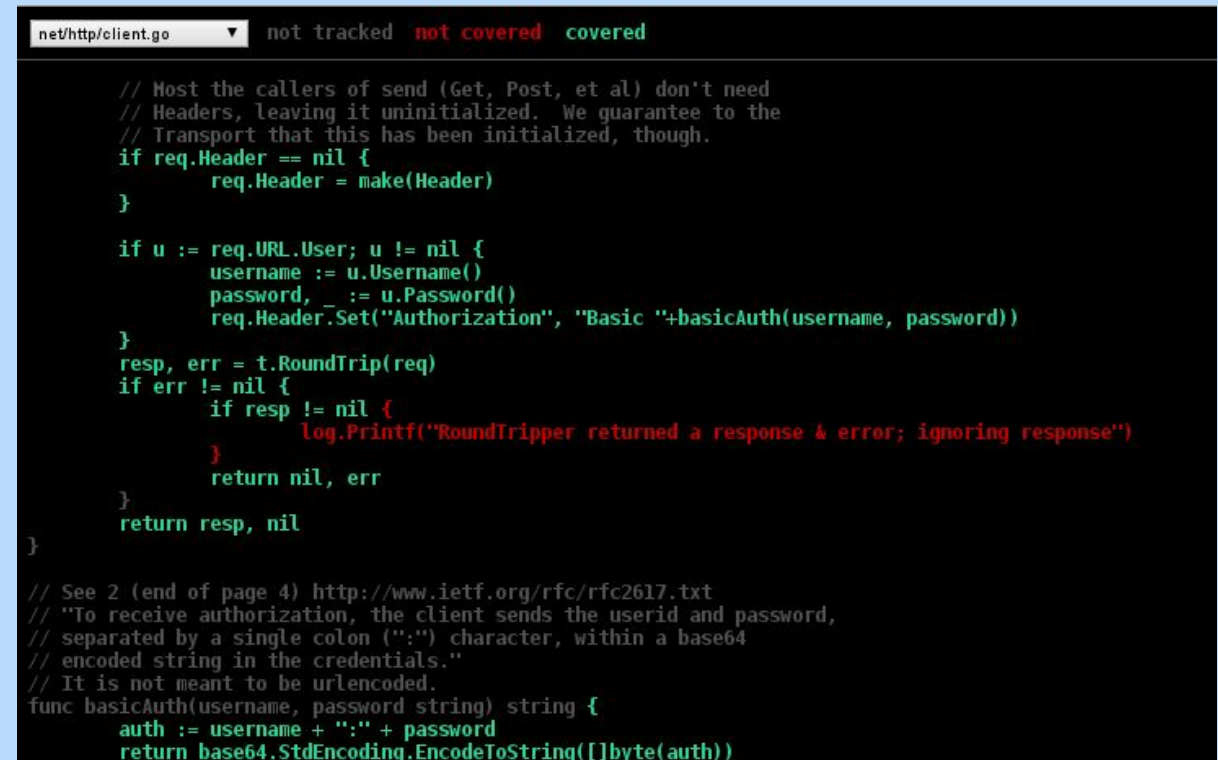
Code coverage

Метрика, которая показывает, какой процент строк/блоков/функций вашего кода был выполнен во время прогона тестов. Позволяет найти непротестированные участки кода.

Запуск: `go test -coverprofile=coverage.out`

Визуализация: `go tool cover -html=coverage.out`

Откроется браузер с подсветкой покрытого (зеленый) и непокрытого (красный) кода.



```
net/http/client.go  not tracked  not covered  covered

// Most the callers of send (Get, Post, et al) don't need
// Headers, leaving it uninitialized. We guarantee to the
// Transport that this has been initialized, though.
if req.Header == nil {
    req.Header = make(Header)
}

if u := req.URL.User; u != nil {
    username := u.Username()
    password, _ := u.Password()
    req.Header.Set("Authorization", "Basic "+basicAuth(username, password))
}
resp, err = t.RoundTrip(req)
if err != nil {
    if resp != nil {
        log.Printf("RoundTripper returned a response & error; ignoring response")
    }
    return nil, err
}
return resp, nil
}

// See 2 (end of page 4) http://www.ietf.org/rfc/rfc2617.txt
// "To receive authorization, the client sends the userid and password,
// separated by a single colon (":") character, within a base64
// encoded string in the credentials."
// It is not meant to be urlencoded.
func basicAuth(username, password string) string {
    auth := username + ":" + password
    return base64.StdEncoding.EncodeToString([]byte(auth))
}
```

Мутационное тестирование

Мутационное тестирование — способ проверить не сам код, а насколько хорошо наши тесты находят ошибки.

В исходный код автоматически вносятся небольшие изменения — **мутации**, имитирующие типичные ошибки разработчика.

Например:

- Изменение условия (было `a > b`, стало `a >= b`)
- Инверсия сравнения (было `a == b`, стало `a != b`)
- Подмена результата (было `return x`, стало `return 0`)
- и другие

После каждой мутации запускаются тесты:

- **Тест упал → мутант убит** — тесты заметили ошибку.
- **Тесты прошли → мутант выжил** — возможно, в тестах не хватает проверки.

Высокий процент покрытия кода ещё не означает хорошие тесты. Мутационное тестирование проверяет, действительно ли тесты способны обнаруживать ошибки.

В АВИТО мы используем

<https://github.com/avito-tech/go-mutesting>



Нагрузочное тестирование

Проверяет работу системы при большом количестве одновременных действий.

Оно помогает понять:

- сколько пользователей выдерживает система;
- как быстро она отвечает под нагрузкой;
- при какой нагрузке появляются ошибки;
- какой компонент становится узким местом.

Как проходит нагрузочное тестирование

В Авито мы используем
<https://github.com/yandex/yandex-tank>

Создаём сценарий

Например: открыть сайт, найти товар, добавить его в корзину и оформить заказ.

Задаём нагрузку

Количество пользователей постепенно увеличивается, например со 100 до 5 000.

Запускаем тест

Инструмент отправляет запросы, а команда наблюдает за системой.

Анализируем результат

Ищем медленные запросы, ошибки и перегруженные компоненты.

Основные показатели

время ответа, RPS, количество
ошибок, CPU и память

Качество кода

VI. Линтеры

- 01 Что такое линтеры и формтеры
- 02 Golangci-lint
- 03 Советы по использованию линтеров
- 04 Go Uber style guide

Что такое линтер

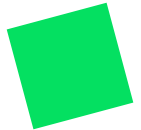
Это инструмент статического анализа, который ищет проблемы в коде:



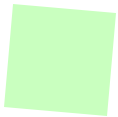
потенциальные баги



плохие практики



неэффективные конструкции



нарушения соглашений



мёртвый код

Что такое формater

Это инструмент, который приводит код к единому стилю:



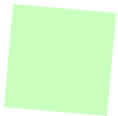
отступы



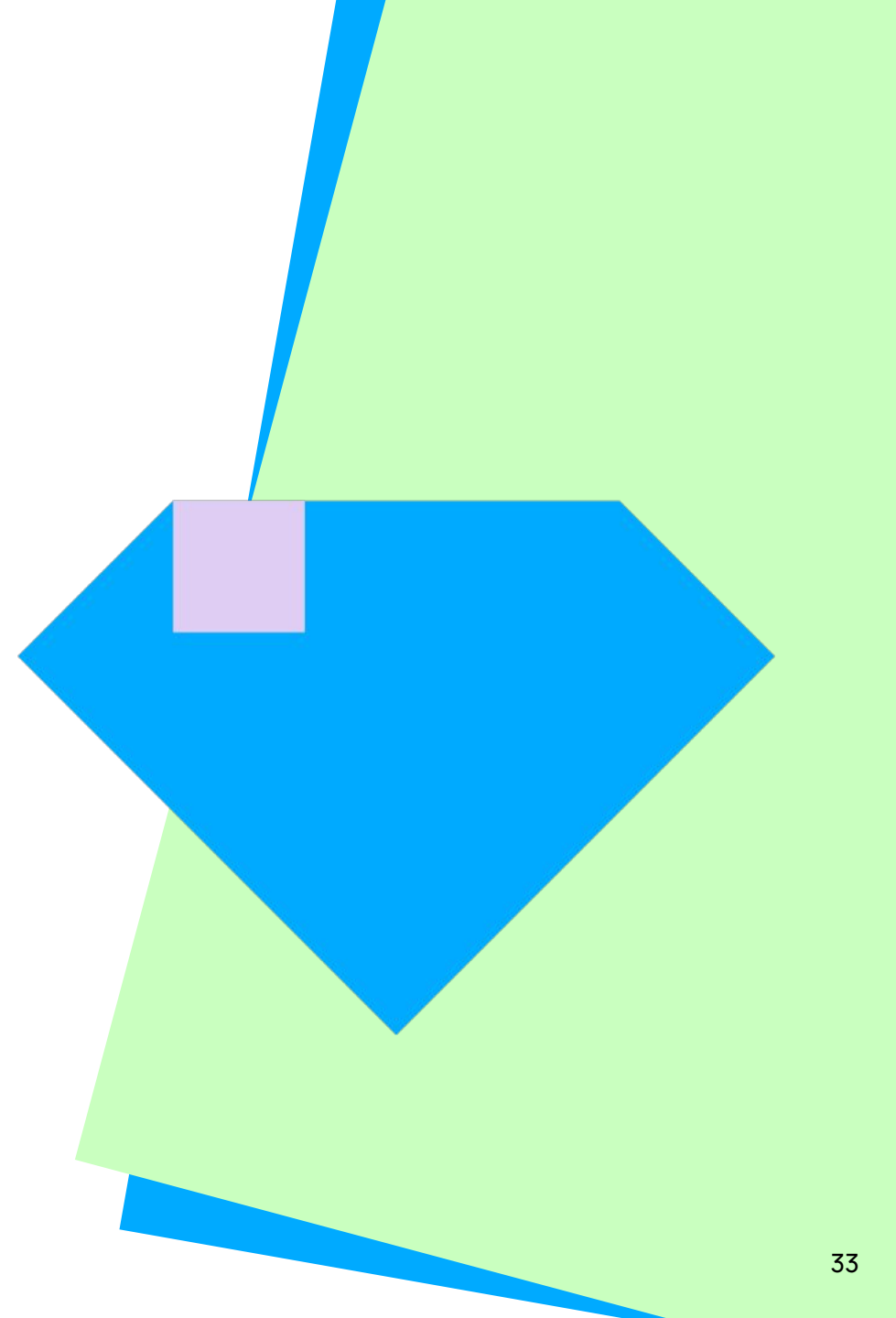
переносы строк



расстановка пробелов



порядок импортов



01

Весь код был
написан
в едином
стиле

02

Быстрее
проходили
code review

Линтеры
и форматтеры
нужны, чтобы:

03

Повысить
качество
кода

golangci-lint

Инструмент, объединяющий в себе множество линтеров и форматтеров. Де-факто стандарт для Go.

golangci-lint.run/docs/linters/



Дока, где можно
посмотреть
про все линтеры

Настройка golangci-lint

Настройки хранятся в файле
.golangci.yml

Запуск: golangci-lint run

Часть линтеров могут исправить
ошибки автоматически: флаг -fix

```
version: "2"
run:
  timeout: 3m
  concurrency: 4
  skip-dirs:
    - testdata
    - generated
  linters:
    default: none
    enable:
      - errcheck
      - gosimple
  settings:
    gocyclo:
      min-complexity: 15
  issues:
    max-issues-per-linter: 10
    new: true
```

Некоторые популярные линтеры

govet Ищет подозрительные конструкции и потенциальные баги

staticcheck Выявляет логические ошибки, устаревшие конструкции и анти-паттерны

errcheck Находит проигнорированные возвращаемые ошибки

ineffassign Обнаруживает присваивания, результат которых не используется

unused Находит неиспользуемые переменные, функции, типы и константы

gosimple Предлагает упрощения кода с сохранением поведения

revive Мощный линтер стиля и best practices

gocognit Оценивает когнитивную сложность функций

dupl Ищет дублирующиеся фрагменты кода

Некоторые популярные линтеры

funcorder

Проверяет порядок объявления функций и методов в файле

bodyclose

Находит забытые вызовы `resp.Body.Close()`

goconst

Предлагает вынести повторяющиеся литералы в константы

godoclint

Проверяет корректность GoDoc-комментариев

gosec

Ищет потенциальные уязвимости безопасности

makezero

Подсказывает использование zero value вместо make, где это возможно

wsl_v5

Контролирует пробелы и пустые строки для читаемости кода

paralleltest

Проверяет использование `t.Parallel()` в тестах

unparam

Находит неиспользуемые параметры функций

Форматтеры

gofmt	Стандартный форматтер Go
gofumpt	Более строгая версия gofmt с дополнительными стилевыми правилами
gci	Сортирует и группирует import'ы по заданным правилам
golines	Автоматически разбивает длинные строки на несколько

Демо

Как внедрить линтеры в большой проект

1. Подключайте линтеры постепенно —

это позволит избежать необходимости массово исправлять существующий код

2. Настройте проверку так, чтобы ошибки фиксировались только в новом коде

(issues: new: true)

3. В отдельных случаях можно осознанно игнорировать срабатывания линтера,

используя директиву `//nolint`

Go uber style guide

Это набор советов и практик, которые помогают писать понятный, надёжный и эффективный код

<https://github.com/uber-go/guide/blob/master/style.md>

Примеры правил:

Интерфейсы:

описывайте интерфейсы там, где они используются (consumer-side), а не там, где реализуются

Работа с Errors:

- используйте %w для оборачивания ошибок (fmt.Errorf)
- проверяйте ошибки сразу (fail fast) — избегайте глубокой вложенности if

Производительность:

- всегда указывайте cap (ёмкость) при make(), если размер слайса известен
- используйте strconv вместо fmt для частого конвертирования типов

Naked Returns:

запрещены (кроме очень коротких функций), так как они ухудшают читаемость

Инициализация:

избегайте использования panic в библиотечном коде — возвращайте ошибки

Полезные материалы

Для тех, кто хочет разобраться глубже.



[Testcontainers for Go](#)

Интеграционные тесты с настоящими PostgreSQL, Redis и другими зависимостями.



[Fuzzing в Go](#)

Поиск входных данных, на которых программа падает или нарушает инварианты.



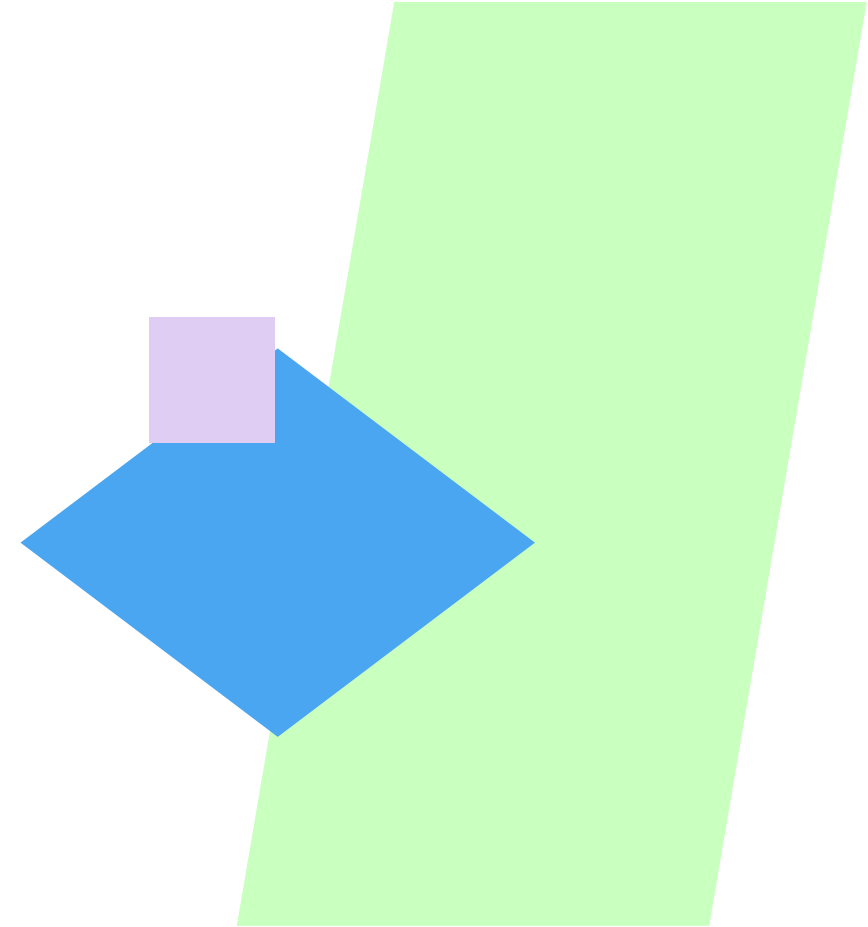
[Практическая пирамида тестирования](#)

Как распределять проверки между unit-, интеграционными и E2E-тестами.



[Flaky Tests at Google](#)

Причины нестабильных тестов и работа с ними в больших проектах.



Сегодня вы узнали:

- 1 Как покрыть код unit и интеграционными тестами
- 2 Как работать с библиотеками testify, httptest
- 3 Почему важно внедрять зависимости через интерфейсы
- 4 Как генерировать моки с помощью mockgen
- 5 Что такое codecoverage и как его посчитать
- 6 Как использовать линтеры и формatters

Полезные материалы:

- 1 Как покрыть код unit и интеграционными тестами
- 2 Как работать с библиотеками testify, httptest
- 3 Почему важно внедрять зависимости через интерфейсы
- 4 Как генерировать моки с помощью mockgen
- 5 Что такое codecoverage и как его посчитать
- 6 Как использовать линтеры и формтеры